Clean Code Guide

For DSA and Web Development — With Real Examples

This guide helps you write code that is easy to read, easy to fix, and easy to share with others. Every section shows you a BAD example and a GOOD example so you clearly understand what to do and what to avoid.

1. Core Principles of Clean Code

What does 'Clean Code' actually mean?

Clean code means writing code that another person (or your future self) can read and understand without spending a lot of time figuring out what it does. Think of it like writing a letter — if the reader has to guess your meaning, you have not written clearly.

Rule 1 — Write for Humans, Not Just Computers

Computers can run any code as long as the syntax is correct. But humans need to understand the code too — to fix bugs, add features, or review it. Always write code that a person can easily read.

BAD — Hard to understand:

```
def f(x): return x*x+2*x+1
```

GOOD — Clear and readable:

```
def calculateSquaredValue(number): return number * number + 2 * number + 1
```

Remember: The GOOD version tells you exactly what is happening. The name 'calculateSquaredValue' explains the purpose instantly.

Rule 2 — One Function, One Job

A function should do ONE thing only. If your function is doing many different things (like fetching data AND formatting it AND saving it), split it into smaller functions. This makes it much easier to find bugs and fix them.

BAD — One function doing too many things:

```
def processUser(u): # fetch from DB, format name, send email — all in one!
```

GOOD — Each function has one clear job:

```
def fetchUserFromDatabase(userId) ... def formatUserDisplayName(user) ... def
sendWelcomeEmail(user) ...
```

Remember: If you can describe your function using the word 'AND', it is doing too much. Split it up.

Rule 3 — Avoid Clever Tricks

Writing code that looks 'smart' but is hard to understand is actually bad code. Simple and clear is always better than clever and confusing.

BAD — Clever one-liner, hard to read:

```
result = [x for x in range(100) if x%2==0 and x%3==0]
```

GOOD — Easy to follow step by step:

```
allNumbers = range(100) result = [] for number in allNumbers: isDivisibleByTwo =
number % 2 == 0 isDivisibleByThree = number % 3 == 0 if isDivisibleByTwo and
```

```
isDivisibleByThree: result.append(number)
```

2. Variable Naming Rules

Why does naming matter?

Variable names are like labels on boxes. If a box says 'stuff', you have to open it to know what's inside. If it says 'userEmailList', you know immediately. Good names save time and prevent bugs.

Rule 1 — Use Descriptive Names

BAD — You have no idea what 'x', 'd', or 'n' mean:

```
x = 5 d = getUserData() n = len(d)
```

GOOD — The names explain themselves:

```
maxRetryCount = 5 userProfileData = getUserData() totalUserCount = len(userProfileData)
```

Remember: When you come back to your code after 2 weeks, you will thank yourself for using clear names.

Rule 2 — Use Plural Names for Lists/Collections

When a variable holds multiple items (a list, array, or set), always use a plural name. This tells the reader right away that it contains many items, not just one.

BAD — Looks like a single item:

```
user = [getUserData(1), getUserData(2), getUserData(3)]
```

GOOD — Clearly shows it holds many users:

```
users = [getUserData(1), getUserData(2), getUserData(3)]
```

Rule 3 — Boolean Variables Should Ask a Question

Boolean variables are either true or false. Name them so they sound like a yes/no question. Use prefixes like: is, has, can, should.

BAD — What does 'login' or 'admin' mean? True or false?

```
login = True admin = False load = True
```

GOOD — Reads like a question, answers itself:

```
isLoggedIn = True isAdminUser = False isLoading = True hasPermission = True canEditProfile = False
```

Remember: If you read 'isLoggedIn = True', it makes sense in plain English. Always aim for that.

Rule 4 — Avoid Vague Names

Names like 'temp', 'data', 'value', 'info', 'stuff', 'thing' say nothing. Every variable in your code should explain what it stores.

BAD — What is 'data'? What is 'temp'?

```
data = fetchFromAPI() temp = data['result'] value = temp * 1.18
```

GOOD — Every name tells a story:

```
productList = fetchFromAPI() rawPrice = productList['result'] priceWithTax = rawPrice * 1.18
```

3. Function Naming Rules

Why function names matter?

A function name is a promise. It tells the reader what the function will do. If you name it well, the reader does not need to read the code inside to understand its purpose.

Rule 1 — Use Action Words (Verbs)

Functions DO things, so their names should start with a verb — a word that describes an action. Good verbs: get, fetch, calculate, validate, send, update, delete, check, build, format.

BAD — Sounds like a noun, not an action:

```
def userData(): ... def totalPrice(): ... def emailCheck(): ...
```

GOOD — Each name is a clear action:

```
def fetchUserData(): ... def calculateTotalPrice(): ... def
validateEmailFormat(): ...
```

> *Remember: When you call a function, ask yourself — 'What is this doing?' The answer should match the name.*

Rule 2 — Be Specific in the Name

Generic names like 'process()', 'handle()', 'doStuff()' are useless. They could mean anything. Be specific about what the function actually does.

BAD — What does 'process' even do here?

```
def process(data): ... def handle(event): ... def run(x): ...
```

GOOD — Now you know exactly what each function does:

```
def processPaymentTransaction(orderData): ... def
handleUserLoginEvent(loginEvent): ... def runMonthlyReportGenerator(): ...
```

Rule 3 — DSA Function Name Examples

When solving coding problems (DSA), name your functions to describe the algorithm or step you are performing.

GOOD Examples for DSA functions:

```
def mergeSortedArrays(leftArray, rightArray): ... def
findMaxSumSubarray(numbers, windowSize): ... def
detectCycleInLinkedList(headNode): ... def binarySearchTarget(sortedArray,
targetValue): ...
```

4. Formatting Rules

What is code formatting?

Formatting is how your code looks — the spaces, blank lines, and indentation. Even if the code runs correctly, bad formatting makes it very hard to read. Think of formatting like paragraphs in a book — without them, the text is a mess.

Rule 1 — Use Consistent Indentation (4 spaces)

Indentation shows which code belongs inside a block (like inside a loop or function). Always use 4 spaces. Never mix spaces and tabs.

BAD — Mixed indentation, hard to follow:

```
def checkAge(age): if age >= 18: print('adult') else: print('minor')
```

GOOD — Consistent 4-space indentation:

```
def checkAge(age): if age >= 18: print('adult') else: print('minor')
```

Rule 2 — Separate Logical Blocks with Blank Lines

Group related lines of code together. Then leave a blank line between different groups. This is exactly like writing paragraphs — each paragraph is one idea.

BAD — Everything crammed together:

```
userName = input('Enter name') userAge = input('Enter age') db.connect()
db.save(userName, userAge) db.close() email.send(userName)
```

GOOD — Logical groups separated clearly:

```
userName = input('Enter name') userAge = input('Enter age') db.connect()
db.save(userName, userAge) db.close() email.send(userName)
```

Remember: Blank lines are FREE. Use them to show where one idea ends and another begins.

Rule 3 — Avoid Deeply Nested Code

When you put an if inside a loop inside another if inside a function, the code becomes a pyramid that is very hard to follow. Try to reduce nesting by returning early.

BAD — Deep nesting, hard to track:

```
def processOrder(order): if order: if order.isValid: if order.user.isLoggedIn: if
order.items: completeOrder(order)
```

GOOD — Return early to reduce nesting:

```
def processOrder(order): if not order: return if not order.isValid: return if not
order.user.isLoggedIn: return if not order.items: return completeOrder(order)
```

Remember: Each level of nesting adds mental load. Flatter code is easier to read and debug.

5. DSA Naming Cheat Sheet

When solving Data Structures and Algorithms problems, use these naming patterns. They make your solution much easier to understand during interviews and code reviews.

Two Pointers Pattern

GOOD variable names for two pointers:

```
leftIndex = 0 rightIndex = len(array) - 1 while leftIndex < rightIndex:
currentSum = array[leftIndex] + array[rightIndex] if currentSum == targetValue:
return [leftIndex, rightIndex]
```

Sliding Window Pattern

GOOD variable names for sliding window:

```
windowStart = 0 windowEnd = 0 windowSum = 0 maxWindowSum = 0 while windowEnd <
len(numbers): windowSum += numbers[windowEnd] if windowEnd - windowStart + 1 ==
windowSize: maxWindowSum = max(maxWindowSum, windowSum) windowSum -=
numbers[windowStart] windowStart += 1 windowEnd += 1
```

Linked List Pattern

GOOD variable names for linked list traversal:

```
currentNode = headNode previousNode = None while currentNode is not None:
nextNode = currentNode.next currentNode.next = previousNode previousNode =
currentNode currentNode = nextNode
```

Tree Traversal Pattern

GOOD variable names for tree traversal:

```
def inorderTraversal(rootNode): if rootNode is None: return [] leftResult =
inorderTraversal(rootNode.leftChild) rightResult =
inorderTraversal(rootNode.rightChild) return leftResult + [rootNode.value] +
rightResult
```

Remember: Consistent naming helps you re-read your solution during an interview without confusion.

6. Web Development Naming Cheat Sheet

These naming patterns are used in professional React and Node.js projects. Following them makes your code look professional and team-friendly.

React Components — Use PascalCase (First Letter of Each Word is Capital)

BAD — Lowercase or unclear names:

```
function userprofile() ... function card() ... function btn() ...
```

GOOD — PascalCase, clearly describes what each component shows:

```
function UserProfileCard() ... function ProductListItem() ... function
CheckoutButton() ...
```

React Hooks — Always Start with 'use'

Custom hooks must start with 'use'. This is a React rule. It also makes the hook instantly recognizable in your codebase.

BAD — Does not follow React hook rules:

```
function fetchUserData() ... function authCheck() ...
```

GOOD — Always starts with 'use':

```
function useFetchUserData() ... function useAuthStatus() ... function
useCartItems() ...
```

API Routes — Use Hyphens, Be Descriptive

BAD — Too short, unclear what action is performed:

```
/user /order /p
```

GOOD — Clear, consistent REST-style routes:

```
GET /api/get-user-profile POST /api/create-new-order PUT
/api/update-user-address DELETE /api/delete-product
```

Constants — Use UPPER_SNAKE_CASE

Constants are values that never change in your app (like API keys, limits, base URLs). Writing them in ALL_CAPS with underscores is the universal way to say 'this value never changes'.

BAD — Looks like a normal variable, could be confused:

```
maxRetry = 3 apiUrl = 'https://api.example.com' timeout = 5000
```

GOOD — UPPER_SNAKE_CASE signals 'this is a constant':

```
MAX_RETRY_COUNT = 3 API_BASE_URL = 'https://api.example.com' REQUEST_TIMEOUT_MS =
5000
```

Remember: When you see UPPER_CASE in any codebase, you immediately know it is a constant that should not be changed.

7. The Clean Code Writing Process

You do not write clean code in one shot. Clean code is the result of writing, then improving. Here is a simple 5-step process you should follow every time you write code.

Step 1 — Write a Working Solution First

Do not worry about names or formatting at first. Just focus on making it work. Use whatever variable names come to mind. Get the logic right first.

First draft — messy but working:

```
def f(a, b): r = 0 for x in a: if x > b: r += x return r
```

Step 2 — Rename Variables for Clarity

Once the code works, go back and rename every variable and function to be descriptive. This is the most important step.

After renaming — same logic, much clearer:

```
def sumNumbersAboveThreshold(numbers, thresholdValue): totalSum = 0 for currentNumber in numbers: if currentNumber > thresholdValue: totalSum += currentNumber return totalSum
```

Step 3 — Add Blank Lines Between Logical Groups

Look at your code and find where one idea ends and another begins. Add a blank line between them, just like paragraphs.

Step 4 — Remove Unnecessary Comments

If your code is well-named, you do not need comments to explain what it does. Remove comments that just repeat what the code already says clearly.

BAD — Comment says exactly what the code already says:

```
# increment counter by 1 counter = counter + 1 # multiply price by tax rate finalPrice = price * taxRate
```

GOOD — Good naming removes the need for obvious comments:

```
counter = counter + 1 priceWithTax = basePrice * TAX_RATE
```

When IS a comment useful? When explaining WHY you made a decision, not WHAT the code does. Example: # Using binary search here because array is always pre-sorted (O log n is faster for large datasets)

Step 5 — Review It Like a Stranger

After writing and cleaning, read your code as if you are seeing it for the first time. Ask yourself: 'Can I understand what this does in 10 seconds?' If yes, you are done. If no, clean it more.

Remember: Clean code is not written — it is rewritten. Every great programmer refactors their code after the first draft.

Quick Memory Card — Remember These Always

#	Rule	Quick Reminder
1	Write for humans	If a stranger can't read it in 10 sec, rename it
2	One function = one job	If you say 'AND', split the function
3	Use descriptive names	No: x, d, temp. Yes: userAge, totalPrice, isLoading
4	Booleans ask questions	isLoggedIn, hasPermission, canEdit, shouldReload
5	Functions use verbs	fetchUser, calculateTotal, validateEmail
6	Use 4-space indent	Never mix tabs and spaces
7	Add blank lines	One blank line = one new idea/group
8	Avoid deep nesting	Return early to keep code flat
9	Constants in CAPS	MAX_COUNT, API_URL, TIMEOUT_MS
10	Refactor always	Write first, clean second — every time

Every time you write code, ask: Can a stranger read this and understand it in 10 seconds?